Documentation-Driven Threat Analysis

BA to Mermaid Visual Guide

Working R1 - costruzione incrementale delle regole grafiche

Dalla Base Analysis alla rappresentazione Mermaid

La guida viene costruita un elemento alla volta attraverso casi di prova usati per comprendere e stabilizzare la projection. Non definisce nuova semantica BA: registra soltanto convenzioni grafiche generalizzabili. Gli esempi della guida restano methodology-neutral e non dipendono dal progetto usato per scoprirle.

Regola editoriale della guida

Ogni elemento o costrutto grafico BA/Mermaid inizia su una **pagina separata**. I blocchi sono mantenuti autocontenuti affinche', al termine della revisione, le pagine possano essere riordinate seguendo lo stesso ordine della Base Analysis Guide senza riscrivere il contenuto.

Stato	WORKING / INCREMENTAL
Metodo di costruzione	Incrementale, mediante casi di prova ricostruiti dalla BA; gli esempi pubblicati nella guida sono generalizzati e project-independent.
Renderer target iniziale	Mermaid flowchart, verificato dall'utente in VS Code.
Regola di authority	La grafica rappresenta la BA; non modifica ne' completa la BA.

Baseline grafica del renderer

La stessa sorgente Mermaid puo' essere visualizzata dentro applicazioni con temi chiari o scuri. Per evitare che lo sfondo o i colori di default cambino in base al programma, ogni diagramma DDTA/BA deve partire da un **tema base esplicito**. Questa regola e' tecnica: non introduce classi o semantica BA.

MERMAID-BASE-01 - renderer baseline

Usare `theme: base` e impostare esplicitamente almeno sfondo, testo di default, bordo di default, colore delle linee ed edge-label background. Gli elementi con significato grafico specifico ricevono poi una propria `classDef`; il tema dell'editor non deve determinarne l'aspetto.

```
%%{init: {
  "theme": "base",
  "themeVariables": {
    "background": "#FFFFFF",
    "primaryColor": "#FFFFFF",
    "primaryTextColor": "#17324D",
    "primaryBorderColor": "#5B6770",
    "lineColor": "#5B6770",
    "edgeLabelBackground": "#FFFFFF"
  }
}}%%
```

Baseline non equivale a semantic style

Il colore/shape di default serve soltanto come fallback stabile. Non deve essere interpretato come stile canonico del BAReferent visualizzato. Quando una famiglia visuale viene definita, la relativa `classDef` sostituisce il fallback.

Traceability BA a bassa intrusivita'

Questa e' una **regola generale di projection** e non appartiene a uno specifico operatore. Quando una visual occurrence deriva da una `BAProposition` inclusa nella view, il relativo identificatore puo' essere preservato nel nodo senza appesantire la lettura umana.

MERMAID-TRACE-01 - trace tag a bassa intrusivita'

Quando una occurrence viene usata come anchor visuale di una `BAProposition`, l'ID della proposition viene riportato **sotto la label canonica**, su una seconda riga, con font molto piccolo e colore **bianco** (`#FFFFFF`). Il tag resta secondario rispetto alla label principale: la bassa intrusivita' deriva dalla dimensione ridotta, non dalla modifica del nome canonico o dalla scelta di un colore differente per ogni fill.

```
ExampleElement["ExampleElement<br/><span style='font-size:7px;color:#FFFFFF'>BAP-EX
    ↪ -01</span>"]
```

Posizione e colore del trace tag

Il trace tag viene mantenuto sotto la label principale e usa sempre `#FFFFFF`. Le regole dei singoli operatori stabiliscono quale occurrence funge da anchor della proposition. Se il fill e' molto chiaro o bianco il tag puo' diventare poco o per nulla visibile: la sua authority resta nel sorgente Mermaid, mentre la leggibilita' umana primaria appartiene alla label canonica.

Il trace tag appartiene alla occurrence, non all'identita'

Il codice `BAP-EX-01` identifica la proposition visualizzata. Non diventa parte del nome canonico o dell'identita' di `ExampleElement`. La regola descrive una annotazione di traceability della occurrence.

Caso con piu' proposition

La forma definitiva per una occurrence che contribuisce contemporaneamente a piu' `BAProposition` resta da verificare. Non si accumulano automaticamente piu' codici nel nodo finche' un caso reale non permette di stabilire una regola leggibile e deterministica.

Visibilita' umana e machine readability non sono equivalenti

Un tag molto piccolo, e in particolare un tag bianco su fill molto chiari, puo' restare disponibile nel sorgente Mermaid o nel testo di un SVG/PDF vettoriale senza essere facilmente leggibile in una rasterizzazione o in uno screenshot. La traceability primaria resta quindi nel modello/sorgente; il trattamento visuale serve soltanto a non perdere il riferimento nella projection.

Scope cumulativo della projection

La projection Mermaid deve dichiarare esplicitamente quali analisi BA sono state incluse nel grafico. La dichiarazione e' una nota di provenance del sorgente e non viene visualizzata nel diagramma.

MERMAID-SCOPE-01 - dichiarazione dello scope BA

Nel file Markdown, immediatamente prima del blocco Mermaid, inserire un commento HTML non renderizzato che elenca le analisi BA incluse e indica se il grafico e' completo rispetto a quello scope. Il commento di provenance resta quindi esterno al parser del diagramma. Quando l'analisi viene estesa a un nuovo livello o ramo, il grafico precedente viene preservato come snapshot del proprio scope e si deriva un **nuovo file cumulativo** che dichiara lo scope esteso.

````
<!--
DDTA PROJECTION SCOPE
BA analyses included: MR-EX
Scope status: complete for MR-EX
When extending the analysis, derive a new file from this graph
and declare the cumulative scope, e.g. MR-EX + DEC-EX.
-->

```mermaid
...
```
````

Provenance esterna al parser Mermaid

Nel renderer target verificato, i commenti Mermaid `%%` usati per la provenance possono comparire come contenuto grafico spurio. Per lo scope DDTA si usa quindi il commento HTML del file Markdown. La direttiva tecnica `%%{init: ...}%%` resta invece all'interno del blocco Mermaid quando necessaria al renderer.

Completezza relativa allo scope dichiarato

La parola "complete" non significa che il sistema sia completamente analizzato. Significa soltanto che la projection contiene tutte le proposition BA accettate appartenenti alle analisi elencate nel commento di scope. Un grafico successivo puo' quindi essere costruito per inclusione, per esempio `MR-EX` → `MR-EX + DEC-EX` → `MR-EX + DEC-EX + FR-EX`, senza riscrivere retroattivamente il significato dello snapshot precedente.

Non sovrascrivere lo snapshot di uno scope concluso

L'aggiunta di BA proveniente da uno scope successivo non trasforma il file precedente nello stesso grafico "piu' grande". Si crea invece una nuova projection cumulativa derivata dal file precedente, mantenendo nel codice una dichiarazione esplicita delle analisi BA che contribuiscono alla view.

1 hasPart e containment dei BAReferent

VISUAL RULE `semanticOperatorKey = hasPart`

1.1 Premessa semantica

La projection non introduce un tipo grafico specifico in base al nome del referent. Se il modello BA identifica soltanto `BAReferent`, il whole e le part restano membri della stessa famiglia visuale `BAReferent`. La relazione `hasPart` determina invece una **topologia di containment**: il whole puo' diventare il frame/subgraph che contiene le occurrence delle part incluse nella view.

HASPART-P01 - containment senza riclassificazione

La relazione `hasPart(whole, part)` puo' essere proiettata come **containment visuale**. Il whole e la part conservano la propria identita' BA e la stessa famiglia visuale; la profondita' nel containment modifica soltanto il trattamento grafico necessario a rendere leggibile la gerarchia.

Node e subgraph sono due topology della stessa famiglia

Un `BAReferent` senza part visualizzate puo' essere mostrato come nodo arrotondato. Quando nella stessa view vengono mostrate le sue part, la sua occurrence puo' essere espansa a subgraph/frame. Il passaggio da nodo a frame non crea un nuovo semantic kind e non cambia l'identita' del `BAReferent`.

1.2 Profondita' di containment e schiaritura

Per rendere visibile l'annidamento senza introdurre colori semantici differenti, la famiglia `BAReferent` usa un blu elettrico come livello esterno e si schiarisce di **15 punti percentuali verso il bianco per ogni livello di profondita'**. La percentuale e' assoluta rispetto al colore base, non cumulativa rispetto al livello precedente.

HASPART-P02 - depth palette deterministica

Sia d la profondita' di containment, con $d = 0$ per il whole piu' esterno visualizzato. La schiaritura e' $L(d) = \min(15d, 100)\%$. Il fill e' ottenuto miscelando il colore base `#0066FF` verso il bianco secondo $L(d)$. A partire da D7 il fill e' bianco. Lo stroke resta `#0047B3` per preservare la continuita' della famiglia visuale.

Depth	Schiaritura	Fill	Testo
D0	0%	`#0066FF`	bianco
D1	15%	`#267DFF`	bianco
D2	30%	`#4C94FF`	bianco
D3	45%	`#73ABFF`	blu scuro
D4	60%	`#99C2FF`	blu scuro
D5	75%	`#BFD9FF`	blu scuro
D6	90%	`#E5F0FF`	blu scuro
D7+	100%	`#FFFFFF`	blu scuro

1.3 Classi Mermaid della palette

I valori della palette vengono fissati esplicitamente nel sorgente per evitare differenze fra renderer. D0 e' normalmente applicato al frame esterno; D1–D7 possono essere applicati sia a nodi sia a frame in base alla topology della occurrence.

```
classDef BAReferentD0 fill:#0066FF,stroke:#0047B3,
  stroke-width:2.5px,color:#FFFFFF;
classDef BAReferentD1 fill:#267DFF,stroke:#0047B3,
  stroke-width:2px,color:#FFFFFF;
classDef BAReferentD2 fill:#4C94FF,stroke:#0047B3,
  stroke-width:2px,color:#FFFFFF;
classDef BAReferentD3 fill:#73ABFF,stroke:#0047B3,
  stroke-width:2px,color:#17324D;
classDef BAReferentD4 fill:#99C2FF,stroke:#0047B3,
  stroke-width:2px,color:#17324D;
classDef BAReferentD5 fill:#BFD9FF,stroke:#0047B3,
  stroke-width:2px,color:#17324D;
classDef BAReferentD6 fill:#E5F0FF,stroke:#0047B3,
  stroke-width:2px,color:#17324D;
classDef BAReferentD7 fill:#FFFFFF,stroke:#0047B3,
  stroke-width:2px,color:#17324D;
```

1.4 Bordi arrotondati

I `BAReferent` della famiglia di containment usano bordi arrotondati. Per i nodi si usa la sintassi Mermaid arrotondata (`("...")`). Per i subgraph il renderer viene normalizzato con una regola CSS sui cluster:

```
%%{init: {
  "theme": "base",
  "themeCSS": ".cluster rect { rx: 28px !important; ry: 28px !important; }"
}}%%
```

Compatibilita' renderer

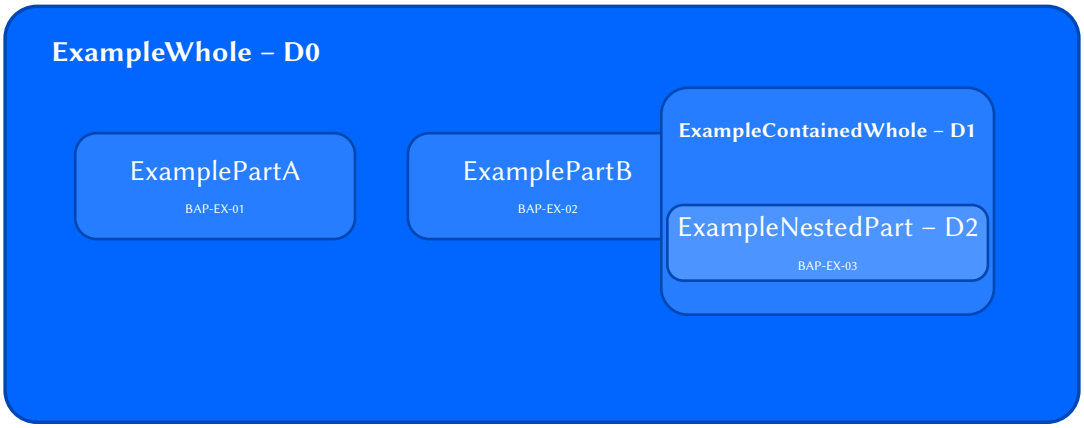
L'arrotondamento dei cluster dipende dal supporto del renderer Mermaid a `themeCSS`. La regola rappresenta il target visuale della guida; la compatibilita' viene verificata nel renderer adottato prima di consolidare una release metodologica.

1.5 Palette visuale D0–D7

La palette seguente e' una rappresentazione visuale della sola profondita' di containment; non corrisponde a differenti semantic kind BA.



1.6 Esempio grafico generico di containment



HASPART-P03 - containment sostituisce l'edge nella composition view

Nella vista di composizione, la presenza della part dentro il frame del whole rappresenta la proposition `hasPart`; non e' necessario aggiungere anche una freccia `hasPart`. Gli ID delle proposition possono essere preservati applicando la regola generale `MERMAID-TRACE-01` alle occurrence interessate.

HASPART-P03A - traceability sulla part

Quando piu' proposition `hasPart` descrivono le part dirette dello stesso whole, il trace tag della singola proposition viene collocato **sotto la label della part corrispondente**. Il whole non ripete tutti gli ID delle relazioni: il containment identifica il whole comune, mentre ogni part conserva il proprio riferimento `BAProposition` secondo `MERMAID-TRACE-01`.

1.7 Esempio Mermaid generico

```
flowchart TB

subgraph ExampleWhole["ExampleWhole"]
  direction TB

  ExamplePartA("ExamplePartA<br/><span style='font-size:7px;color:#FFFFFF'>BAP-EX
    ↪ -01</span>")
  ExamplePartB("ExamplePartB<br/><span style='font-size:7px;color:#FFFFFF'>BAP-EX
    ↪ -02</span>")
```

```
  subgraph ExampleContainedWhole["ExampleContainedWhole"]
    ExampleNestedPart("ExampleNestedPart<br/><span style='font-size:7px;color:#
    ↪ FFFFFF'>BAP-EX-03</span>")
  end
end

classDef BAReferentD1 fill:#267DFF,stroke:#0047B3,
  stroke-width:2px,color:#FFFFFF;
classDef BAReferentD2 fill:#4C94FF,stroke:#0047B3,
  stroke-width:2px,color:#FFFFFF;

class ExamplePartA,ExamplePartB BAReferentD1;
class ExampleNestedPart BAReferentD2;

style ExampleWhole fill:#0066FF,stroke:#0047B3,
  stroke-width:2.5px,color:#FFFFFF
style ExampleContainedWhole fill:#267DFF,stroke:#0047B3,
  stroke-width:2.5px,color:#FFFFFF
```

HASPART-P04 - le relazioni esterne puntano al whole

Quando un `BAReferent` e' espanso a subgraph per mostrare le sue part, una relazione BA che ha quel referent come endpoint deve terminare sul **frame del whole**, non su una part scelta per convenienza grafica. In Mermaid si usa quindi l'ID del subgraph come source o target dell'edge. Il containment non modifica l'endpoint semantico della proposition esterna.

```
ExternalActor -->|operator| ExampleWhole
```

Non dedurre una tassonomia dal livello visuale

D0, D1, D2 e i livelli successivi non sono tipi BA. Descrivono soltanto la profondita' della occurrence nella projection corrente. Lo stesso `BAReferent` puo' apparire a una profondita' diversa in una view con root differente senza cambiare identita' o semantica.

2 produce

VISUAL RULE `semanticOperatorKey = produce`

2.1 Scopo della rappresentazione

La proposition `produce` viene letta graficamente come una funzione che riceve eventuali input, opera attraverso il proprio `actor` e rende disponibili uno o piu' `result`. La grafica di `produce` non assegna lo stile semantico ai result: ogni result conserva o ricevera' la propria regola visuale indipendente.

Esempio generico:

```
BAP-EX-01 - produce
actor  -> ExampleProducer
result -> ExampleResult
```

La lettura funzionale e': `ExampleProducer` produce `ExampleResult`.

2.2 Regola grafica candidata

Parte BA	Projection Mermaid corrente
`actor`	Black box: rettangolo ad angoli retti, sfondo antracite/nero, bordo nero, testo principale bianco.
`produce`	Freccia direzionale uscente dall'actor con label testuale `produce` su fondo bianco, bordo sottile e angoli arrotondati. Non viene creato un nodo autonomo `produce`.
`input`	Quando presente, entra nella black box. La relativa shape dipende dalla regola visuale dell'elemento che occupa il role `input`.
`result`	Esce dalla black box. La relativa shape e il colore **non sono definiti da produce**; dipendono dalla regola visuale propria del result.

PRODUCE-P01 - black box funzionale

L'`actor` di una `produce` viene proiettato come **black box rettangolare ad angoli retti**. La black box comunica una funzione/responsabilita' osservata attraverso input e result senza trasformarla graficamente nella sua realizzazione concreta.

PRODUCE-P02 - edge label arrotondata

La label `produce` resta associata alla freccia e non diventa un nodo BA. Il target visuale usa **fondo bianco, bordo sottile e angoli arrotondati** per separare la label dalla linea senza introdurre una nuova shape semantica. Nel renderer Mermaid corrente il trattamento viene applicato alla edge label tramite `themeCSS`.

```
%%{init: {
  "theme": "base",
  "themeCSS": ".edgeLabel p { background:#FFFFFF !important; border:1px solid #
    ↪ B8CBE8 !important; border-radius:10px !important; padding:2px 7px !important;
    ↪  }"
}}%%
```

2.3 Classe visuale dell'actor

La classe e' nominata per il significato di projection, non per il colore:

```
classDef BAProduceActor \
    fill:#212121, \
    stroke:#000000, \
    stroke-width:2px, \
    color:#FFFFFF;
```

La sintassi rettangolare Mermaid `["..."]` mantiene gli angoli retti; non vengono aggiunti `rx`/`ry`. Nella projection `produce`, l'actor e' anche l'anchor della proposition: il relativo ID `BAProposition` viene quindi riportato sotto il nome dell'actor secondo `MERMAID-TRACE-01`.

2.4 Esempio Mermaid minimo

Il result `ExampleResult` e' lasciato intenzionalmente al solo fallback della baseline: la sua forma canonica dipende dalla regola visuale dell'elemento/famiglia che occupa il role `result` e **non** dalla regola `produce`.

Applicare prima la baseline `MERMAID-BASE-01`; il blocco specifico della `produce` resta quindi compatto:

```
flowchart LR

ExampleProducer["ExampleProducer<br/><span style='font-size:7px;color:#FFFFFF'>BAP-
    ↪ EX-01</span>"]
ExampleResult["ExampleResult"]

ExampleProducer -->|produce| ExampleResult

classDef BAProduceActor \
  fill:#212121,stroke:#000000,stroke-width:2px,color:#FFFFFF;
class ExampleProducer BAProduceActor;
```

2.5 Esempio grafico visivo

La figura seguente mostra il **target visuale corrente** della sola regola `produce`. Il nodo di destra usa volutamente il fallback della baseline: non assegna al result una forma derivata da `produce`. Il disegno in guida e' un riferimento visuale della convenzione; il sorgente Mermaid resta la forma eseguibile da verificare nel renderer scelto.



Che cosa e' fissato dalla figura

La figura fissa soltanto gli aspetti specifici di `produce`: **black box ad angoli retti** per l'`actor` e freccia uscente con label `produce` su pill bianca arrotondata con bordo sottile. Il trace tag applica la regola generale `MERMAID-TRACE-01`. Forma e stile del result restano intenzionalmente fuori da questa pagina.

Non inferire lo stile del result da produce

Il fatto che un elemento occupi il role `result` non autorizza a renderlo con la stessa black box, con una shape "output" generica o con un colore scelto da `produce`. `produce` governa il pattern di connessione; lo stile del result appartiene alla regola visuale dell'elemento che occupa quel role.

3 dependOn

VISUAL RULE `semanticOperatorKey = dependOn`

3.1 Scopo della rappresentazione

La proposition `dependOn` esprime una dipendenza direzionale fra un `dependent` e un `prerequisite`. La projection deve rendere leggibile la dipendenza senza attribuire ai due referent un nuovo semantic kind e senza competere visivamente con relazioni funzionali piu' strutturanti.

Esempio generico:

```
BAP-EX-02 - dependOn
dependent    -> ExampleDependent
prerequisite -> ExamplePrerequisite
```

La lettura grafica e': `ExampleDependent` dipende da `ExamplePrerequisite`.

3.2 Regola grafica candidata

Parte BA	Projection Mermaid corrente
`dependent`	Sorgente della freccia. Mantiene la shape e lo stile propri del referent. E' l'anchor del trace tag della proposition.
`dependOn`	Freccia direzionale tratteggiata, sottile, in famiglia giallo/ocra. La label e' una pill arrotondata con fondo giallo chiaro, bordo ocra e testo ocra scuro.
`prerequisite`	Destinazione della freccia. Mantiene la shape e lo stile propri del referent. Se e' visualizzato come whole/subgraph, la relazione termina sul whole nel suo complesso.

DEPENDON-P01 - direzione dependent verso prerequisite

La direzione della freccia segue direttamente i role BA: `dependent` → `prerequisite`. La label `dependOn` resta associata all'edge e non diventa un nodo BA autonomo.

DEPENDON-P02 - peso visuale secondario

La relazione usa stroke `#C69214`, spessore `1.5px` e tratteggio `6 4`. Il trattamento leggero rende la dipendenza informativa distinguibile senza farla dominare sulla struttura funzionale o di containment della stessa view.

DEPENDON-P03 - edge label giallo/ocra arrotondata

La label `dependOn` usa fondo `#FFF4CC`, bordo `#C69214`, testo `#7A5A00` e angoli arrotondati. La pill resta una label dell'edge: il trattamento grafico non introduce un nuovo referent.

3.3 Traceability della proposition

Nella projection `dependOn`, il `dependent` e' l'anchor della proposition. Il relativo ID `BAProposition` viene quindi riportato sotto la label del dependent, in piccolo e in bianco, secondo `MERMAID-TRACE-01`.

Il prerequisite non ripete lo stesso ID.

3.4 Esempio Mermaid minimo

L'esempio seguente isola la sola relazione `dependOn`; poiche' contiene una sola edge label, il `themeCSS` puo' applicare direttamente il trattamento giallo/ocra alla label senza influenzare altri operatori.

```
%%{init: {
  "theme": "base",
  "themeCSS": ".edgeLabel p { background:#FFF4CC !important; border:1px solid #
    ↪ C69214 !important; border-radius:10px !important; padding:2px 7px !important;
    ↪  color:#7A5A00 !important; }"
}}%%

flowchart LR

ExampleDependent("ExampleDependent<br/><span style='font-size:7px;color:#FFFFFF'>
    ↪ BAP-EX-02</span>")
ExamplePrerequisite("ExamplePrerequisite")

ExampleDependent -.->|dependOn| ExamplePrerequisite

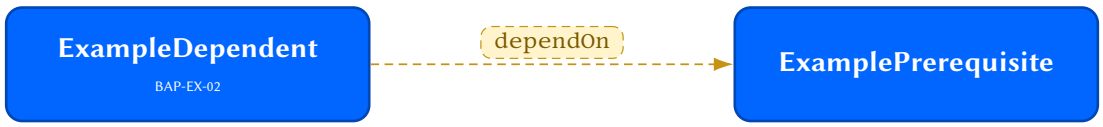
classDef BAReferentD0 fill:#0066FF,stroke:#0047B3,
  stroke-width:2px,color:#FFFFFF;
class ExampleDependent,ExamplePrerequisite BAReferentD0;

linkStyle 0 stroke:#C69214,stroke-width:1.5px,
  stroke-dasharray:6 4;
```

Coesistenza di edge label con stili diversi

In una view composita possono coesistere operatori con edge label differenti, per esempio `produce` e `dependOn`. La regola semantica fissa il target visuale di ciascun operatore; l'eventuale selettore CSS necessario a stilizzare una singola edge label deve essere verificato nel renderer adottato e non deve alterare le label degli altri operatori.

3.5 Esempio grafico visivo



DEPENDON-P04 - il prerequisite conserva la propria topology

Se il prerequisite e' un `BAReferent` espanso a subgraph perche' nella stessa view sono mostrate le sue part, la freccia `dependOn` termina sul frame del whole. Non viene scelta arbitrariamente una part interna come destinazione. Il giallo/ocra appartiene soltanto all'edge e alla label: i referent collegati mantengono shape, fill, depth palette e identita' determinati dalle rispettive regole visuali.

4 realize

VISUAL RULE `semanticOperatorKey = realize`

4.1 Scopo della rappresentazione

La proposition `realize` collega una responsabilita' o funzione astratta alla realizzazione concreta che la attua nella baseline osservata. La projection deve mantenere distinte le due identita': l'`abstract` conserva la propria famiglia visuale, mentre la `realization` viene resa riconoscibile mediante una famiglia grigia dedicata alla relation view.

Esempio generico:

```
BAP-EX-03 - realize
abstract    -> ExampleAbstract
realization -> ExampleRealization
```

La lettura grafica e': `ExampleRealization` realizza `ExampleAbstract`.

REALIZE-P01 - direzione realization verso abstract

La freccia segue direttamente i role BA: `realization` → `abstract`. L'edge usa stroke grigio `#5F6368`, spessore `1.8px` e tratteggio `6 4`. La label `realize` resta associata all'edge e non diventa un nodo BA autonomo.

REALIZE-P02 - famiglia grigia della realization

L'occurrence che occupa il role `realization` usa come livello D0 il fill `#5F6368`. Quando la realization e' espansa per mostrare le proprie part, il whole viene proiettato come subgraph/frame con stroke `#3F4347`, spessore `2.5px` e bordo tratteggiato `6 4`. Il tratteggio del frame identifica la topology di realization nella projection; non introduce un nuovo semantic kind BA.

REALIZE-P03 - depth palette interna alla realization

Se nella stessa view sono mostrate proposition `hasPart` del referent che occupa il role `realization`, le part contenute usano la stessa famiglia grigia e vengono schiarite di **15 punti percentuali verso il bianco per ogni livello di containment**. Le part hanno bordo continuo; il bordo tratteggiato resta sul whole che costituisce l'endpoint della proposition `realize`.

L'abstract conserva il proprio stile

`realize` non riclassifica l'`abstract`. Shape, fill e topology dell'abstract restano determinati dalle altre proposition e dalle altre regole visuali applicabili nella view. La famiglia grigia appartiene alla projection della realization e al suo containment interno.

4.2 Traceability

L'anchor della proposition `realize` e' la `realization`: l'ID della `BAProposition` viene riportato sotto la label del nodo o del subgraph che rappresenta la realization secondo `MERMAID-TRACE-01`. Le proposition `hasPart` interne mantengono invece il proprio trace tag sotto la part corrispondente, secondo `HASPART-P03A`.

4.3 Esempio Mermaid generico

```
%%{init: {
  "theme": "base",
  "themeCSS": ".cluster rect { rx:28px !important; ry:28px !important; }"
}}%%

flowchart LR

ExampleAbstract("ExampleAbstract")

subgraph ExampleRealization["ExampleRealization<br/><span style='font-size:7px;
    ↪ color:#FFFFFF'>BAP-EX-03</span>"]
  direction TB
  ExampleStageA("ExampleStageA<br/><span style='font-size:7px;color:#FFFFFF'>BAP-EX
    ↪ -04</span>")
  ExampleStageB("ExampleStageB<br/><span style='font-size:7px;color:#FFFFFF'>BAP-EX
    ↪ -05</span>")
end

ExampleRealization -.->|realize| ExampleAbstract

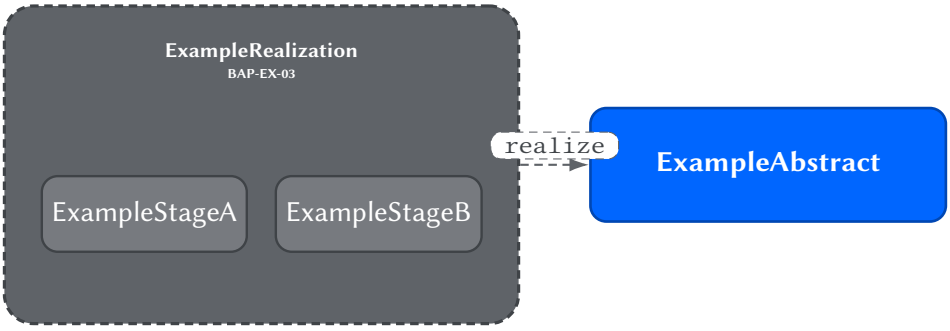
classDef BAReferentD0 fill:#0066FF,stroke:#0047B3,
  stroke-width:2px,color:#FFFFFF;
classDef BARealizationD1 fill:#777A7F,stroke:#3F4347,
  stroke-width:2px,color:#FFFFFF;

class ExampleAbstract BAReferentD0;
class ExampleStageA,ExampleStageB BARealizationD1;

style ExampleRealization fill:#5F6368,stroke:#3F4347,
  stroke-width:2.5px,stroke-dasharray:6 4,color:#FFFFFF

linkStyle 0 stroke:#5F6368,stroke-width:1.8px,
  stroke-dasharray:6 4;
```

4.4 Esempio grafico visivo



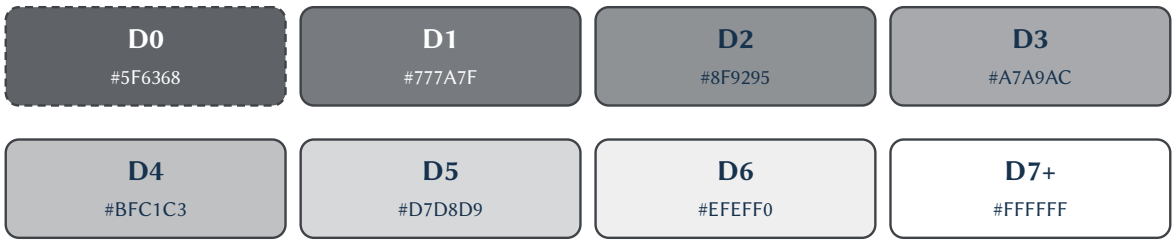
Non propagare il tratteggio alle part

Il bordo tratteggiato non significa “componente tecnico” e non viene ereditato automaticamente dalle part. Le part interne mantengono bordo continuo e ricevono soltanto la schiaritura di depth della famiglia grigia, salvo che una proposition indipendente richieda per esse un’altra projection.

Scala cromatica completa della famiglia `realize`

La palette della realization applica la stessa regola deterministica di schiaritura usata per il containment: a partire dal base fill `#5F6368`, il livello D_d e' ottenuto miscelando il colore base verso il bianco di $L(d) = \min(15d, 100)\%$. La scala e' una proprieta' grafica della projection e non una tassonomia BA.

Depth	Schiaritura	Fill	Testo principale
D0	0%	`#5F6368`	`#FFFFFF`
D1	15%	`#777A7F`	`#FFFFFF`
D2	30%	`#8F9295`	`#17324D`
D3	45%	`#A7A9AC`	`#17324D`
D4	60%	`#BFC1C3`	`#17324D`
D5	75%	`#D7D8D9`	`#17324D`
D6	90%	`#EFEFF0`	`#17324D`
D7+	100%	`#FFFFFF`	`#17324D`



REALIZE-P04 - valori espliciti nel sorgente Mermaid

Per ridurre differenze fra renderer, i valori D0–D7 vengono fissati esplicitamente nel sorgente. D0 e' il whole della realization; D1–D7 sono disponibili per le part annidate in base alla depth effettivamente mostrata nella view.

Mermaid palette test completo

Il blocco seguente rende disponibili tutti i livelli della famiglia `realize` per test di renderer e per view con containment profondo. Il file di prova standalone usa gli stessi valori.

```
classDef BARealizationD0 fill:#5F6368,stroke:#3F4347,
  stroke-width:2.5px,color:#FFFFFF;
classDef BARealizationD1 fill:#777A7F,stroke:#3F4347,
  stroke-width:2px,color:#FFFFFF;
classDef BARealizationD2 fill:#8F9295,stroke:#3F4347,
  stroke-width:2px,color:#17324D;
classDef BARealizationD3 fill:#A7A9AC,stroke:#3F4347,
  stroke-width:2px,color:#17324D;
classDef BARealizationD4 fill:#BFC1C3,stroke:#3F4347,
  stroke-width:2px,color:#17324D;
classDef BARealizationD5 fill:#D7D8D9,stroke:#3F4347,
  stroke-width:2px,color:#17324D;
classDef BARealizationD6 fill:#EFEFF0,stroke:#3F4347,
  stroke-width:2px,color:#17324D;
classDef BARealizationD7 fill:#FFFFFF,stroke:#3F4347,
  stroke-width:2px,color:#17324D;
```

Il trace tag resta bianco

Come stabilito da `MERMAID-TRACE-01`, il trace tag `BAProposition` resta sempre bianco anche ai livelli chiari. Il cambio di colore indicato nella tabella riguarda il testo principale del nodo; la traceability autorevole resta nel sorgente anche quando il tag e' poco visibile.

5 constrain

VISUAL RULE `semanticOperatorKey = constrain`

5.1 Scopo della rappresentazione

La proposition `constrain` collega un significato governato che impone una restriction al `constraintTarget`. Nella BA i role restano espliciti come `constraintTarget` e `constraintValue`; la projection Mermaid rende invece leggibile la frase naturale "il constraint value vincola il target".

Esempio generico:

```
BAP-EX-06 - constrain
constraintTarget -> ExampleEnvironment
constraintValue  -> ExampleConfigurationProfile
polarity = affirmative
```

La lettura grafica e': `ExampleConfigurationProfile` vincola `ExampleEnvironment`.

CONSTRAIN-P01 - direzione constraintValue verso constraintTarget

La freccia di projection e' orientata `constraintValue` → `constraintTarget`. Questa direzione non modifica i role BA: serve a rendere leggibile il predicato naturale "X constrains Y". L'edge usa stroke viola `#6F42C1`, spessore `1.8px` e tratteggio `6 4`.

CONSTRAIN-P02 - famiglia viola del named constraint

Quando il `constraintValue` e' un `BAReferent` nominato e riusabile, la sua occurrence usa la famiglia viola. Il livello D0 usa fill `#6F42C1` e stroke `#55308F`. Se il named constraint e' mostrato come nodo semplice, mantiene la propria identita' di BAReferent; se viene aperto per mostrarne il dettaglio, la stessa identita' diventa un subgraph/frame con bordo tratteggiato `6 4`.

CONSTRAIN-P03 - apertura del named constraint senza hasPart

Una view piu' dettagliata puo' aprire il named constraint e collocare al suo interno card che rappresentano constraint di dettaglio aventi quel profile come `constraintTarget`. Il containment e' una shorthand di projection per un target comune: **non introduce** `hasPart`, non crea un operatore `specialize` e non cambia l'identita' del profile.

CONSTRAIN-P04 - depth palette interna

Le card interne usano la stessa famiglia viola e vengono schiarite di **15 punti percentuali verso il bianco per ogni livello visuale**. D1–D7 usano bordo continuo. Il bordo tratteggiato resta sul whole D0 quando il named constraint e' espanso.

Authoring metadata non proiettato

Campi di review come `constraintForm = NAMED_REFERENT` non sono field BA canonici e non vengono proiettati. La view deriva soltanto dai role e dalle proposition BA accettate.

5.2 Traceability

Per la proposition principale, l'anchor visuale e' il named constraint che occupa il role `constraintValue`: il relativo ID `BAProposition` viene mostrato sotto la label del nodo o del subgraph. Quando il profile e' aperto, ogni card di dettaglio puo' riportare il proprio ID `BAProposition`; il frame comune rende visibile il target condiviso senza richiedere edge interni ripetuti.

5.3 Esempio Mermaid generico

```
%%{init: {
  "theme": "base",
  "themeCSS": ".cluster rect { rx:28px !important; ry:28px !important; }"
}}%%

flowchart LR

ExampleEnvironment("ExampleEnvironment")

subgraph ExampleConfigurationProfile["ExampleConfigurationProfile<br/><span style='
    ↪ font-size:7px;color:#FFFFFF'>BAP-EX-06</span>"]
  direction TB
  MinimumA("minimumA = 16<br/><span style='font-size:7px;color:#FFFFFF'>BAP-EX-07</
    ↪ span>")
  MinimumB("minimumB = 50<br/><span style='font-size:7px;color:#FFFFFF'>BAP-EX-08</
    ↪ span>")
end

ExampleConfigurationProfile -.->|constrain| ExampleEnvironment

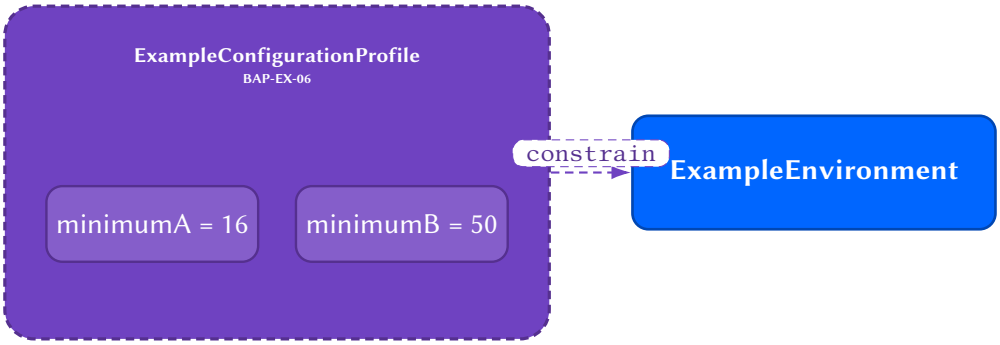
classDef BAReferentD0 fill:#0066FF,stroke:#0047B3,
  stroke-width:2px,color:#FFFFFF;
classDef BAConstrainD1 fill:#855ECA,stroke:#55308F,
  stroke-width:2px,color:#FFFFFF;

class ExampleEnvironment BAReferentD0;
class MinimumA,MinimumB BAConstrainD1;

style ExampleConfigurationProfile fill:#6F42C1,stroke:#55308F,
  stroke-width:2.5px,stroke-dasharray:6 4,color:#FFFFFF

linkStyle 0 stroke:#6F42C1,stroke-width:1.8px,
  stroke-dasharray:6 4;
```

5.4 Esempio grafico visivo



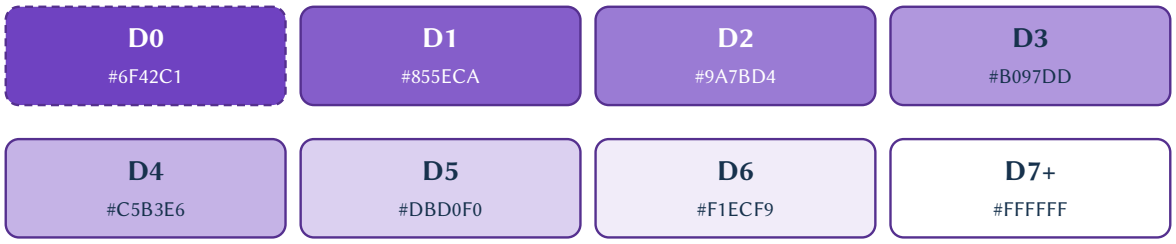
Non leggere il containment come composizione

Le card interne non diventano part del profile per il solo fatto di essere disegnate nel frame. La semantica resta quella delle proposition `constrain` che condividono il profile come target. Se la BA non contiene tali proposition, la view non puo' inventare detail interni.

Scala cromatica completa della famiglia `constrain`

La palette applica la regola deterministica di schiaritura gia' usata dalle altre famiglie: a partire dal base fill `#6F42C1`, il livello D_d e' ottenuto miscelando il colore base verso il bianco di $L(d) = \min(15d, 100)\%$. D0 identifica il named constraint/profile nella view; i livelli successivi sono disponibili per detail annidati.

Depth	Schiaritura	Fill	Testo principale
D0	0%	`#6F42C1`	`#FFFFFF`
D1	15%	`#855ECA`	`#FFFFFF`
D2	30%	`#9A7BD4`	`#FFFFFF`
D3	45%	`#B097DD`	`#17324D`
D4	60%	`#C5B3E6`	`#17324D`
D5	75%	`#DBD0F0`	`#17324D`
D6	90%	`#F1ECF9`	`#17324D`
D7+	100%	`#FFFFFF`	`#17324D`



CONSTRAIN-P05 - valori espliciti nel sorgente Mermaid

Per ridurre differenze fra renderer, i valori D0–D7 vengono fissati esplicitamente. La schiaritura e' una regola della projection e non una tassonomia BA.

Mermaid palette test completo

Il blocco seguente rende disponibili tutti i livelli della famiglia `constrain`. D0 e' predisposto per il whole espanso; D1–D7 usano bordo continuo.

```
classDef BAConstrainD0 fill:#6F42C1,stroke:#55308F,
  stroke-width:2.5px,stroke-dasharray:6 4,color:#FFFFFF;
classDef BAConstrainD1 fill:#855ECA,stroke:#55308F,
  stroke-width:2px,color:#FFFFFF;
classDef BAConstrainD2 fill:#9A7BD4,stroke:#55308F,
  stroke-width:2px,color:#FFFFFF;
classDef BAConstrainD3 fill:#B097DD,stroke:#55308F,
  stroke-width:2px,color:#17324D;
classDef BAConstrainD4 fill:#C5B3E6,stroke:#55308F,
  stroke-width:2px,color:#17324D;
classDef BAConstrainD5 fill:#DBD0F0,stroke:#55308F,
  stroke-width:2px,color:#17324D;
classDef BAConstrainD6 fill:#F1ECF9,stroke:#55308F,
  stroke-width:2px,color:#17324D;
classDef BAConstrainD7 fill:#FFFFFF,stroke:#55308F,
  stroke-width:2px,color:#17324D;
```

Direzione e trace restano semantiche, il colore e' solo projection

Il viola distingue visivamente la famiglia `constrain` da `dependOn`, `realize` e dai BAReferent blu. La direction resta `constraintValue -> constraintTarget`; il trace tag resta bianco secondo `MERMAID-TRACE-01`.

6 boundaryOf e projection del Boundary

VISUAL RULE `semanticKind = boundary; semanticOperatorKey = boundaryOf`

6.1 Scopo della rappresentazione

Un `BAReferent` con `semanticKind = boundary` identifica un confine architetturale/topologico. La proposition `boundaryOf` stabilisce quale scope e' delimitato dal boundary. La projection rende il boundary come **frame visibile attorno alla projection dello scope**; il frame appartiene al Boundary, non allo scope e non introduce composizione.

```
BAP-EX-B01 - boundaryOf
boundary -> ExampleEnvironmentBoundary
scope    -> ExampleEnvironment
polarity = affirmative
```

BOUNDARY-P01 - frame deterministico di boundaryOf

Se la BA inclusa nella view contiene `boundaryOf(boundary=B, scope=S)`, proiettare `B` come frame che circonda la occurrence o il subgraph con cui la stessa view proietta `S`. Il frame riporta la label canonica di `B`; la occurrence di `S` conserva shape, colore e topologia stabiliti dalle proprie regole.

BOUNDARY-P02 - il frame non e' hasPart

Il nesting grafico `Boundary -> scope` e' la projection di `boundaryOf`, non di `hasPart`. Non inferire quindi `Boundary hasPart scope`, ne' che gli elementi contenuti nello scope siano part del Boundary. Se lo scope e' gia' un frame di containment `hasPart`, i due frame restano annidati ma semanticamente distinti.

Boundary neutro rispetto al trust

La BA del Boundary non classifica automaticamente i due lati come trusted/untrusted, sicuri/insicuri o a rischio alto/basso. Il boundary registra il cambio di scope rilevante; eventuali assunzioni di trust, controlli e livelli di rischio appartengono a una successiva analisi di sicurezza.

6.2 Stile visuale di default prima della security analysis

BOUNDARY-P03 - rosso conservativo UNASSESSED

Quando il Boundary non dispone ancora di una valutazione di sicurezza esplicita e tracciabile, usare fill `#FFF8F8`, stroke rosso `#C62828`, spessore `3px`, tratteggio `7 5` e testo `#8B1A1A`. Il rosso e' uno **stato visuale conservativo UNASSESSED**: segnala un potenziale punto di pericolo da analizzare, non significa "untrusted", "compromesso" o "risk = high". Il tratteggio distingue il Boundary dal bordo continuo del containment, senza introdurre una nuova semantica.

```
style ExampleEnvironmentBoundary fill:#FFF8F8,stroke:#C62828,
  stroke-width:3px,stroke-dasharray:7 5,color:#8B1A1A
```

6.3 `crossesBoundary`: overlay su una proposition esistente

VISUAL RULE `semanticOperatorKey = crossesBoundary`

```
BAP-EX-R01 - <underlying relation>
  ...roles of the underlying operator...

BAP-EX-B02 - crossesBoundary
relation -> BAP-EX-R01
boundary -> ExampleEnvironmentBoundary
polarity = affirmative
```

BOUNDARY-P04 - nessuna seconda edge semantica

`crossesBoundary` qualifica una `BAProposition` gia' esistente. Nella projection non creare un nodo o una edge separata per il qualifier: applicare l'overlay di crossing alla edge/topologia con cui viene resa la proposition indicata dal role `relation`. L'operatore sottostante conserva direzione, role, dash pattern e significato.

BOUNDARY-P05 - crossing UNASSESSED evidenziato in rosso

Se una proposition inclusa nella view e' qualificata da `crossesBoundary` e non esiste ancora una security assessment che ne governi lo stato visuale, rendere il tratto relazionale di crossing in rosso `#C62828` con spessore `3px`. Per un transfer gia' proiettato come linea continua, la linea resta continua e diventa rossa; per altri operatori si conserva il pattern definito dalla loro regola e si applica soltanto l'enfasi di crossing.

Il rosso non sostituisce la security analysis

Boundary e crossing restano rossi finche' la view non include una valutazione di sicurezza esplicita. Una futura BA/security overlay potra' governare stati differenti — per esempio giallo o verde — ma soltanto mediante un contract tracciabile che definisca significato e criteri. Questa guida non assegna soglie di rischio ne' deduce un colore favorevole dalla sola presenza di controlli.

BOUNDARY-P06 - traceability dei due operatori

L'ID della proposition `boundaryOf` e' ancorato al frame del Boundary sotto la label canonica. L'ID della proposition `crossesBoundary` e' ancorato alla relation qualificata, in una riga secondaria molto piccola dell'edge label quando tale label e' inclusa nella view. Questo caso specifico risolve la trace di una proposition qualificante senza creare una seconda edge.

Non dedurre il crossing dalla sola geometria

Una edge che nel layout Mermaid sembra attraversare un frame non acquisisce `crossesBoundary`. Viceversa, una proposition `crossesBoundary` resta autorevole anche se il routing automatico del renderer rende il punto geometrico di attraversamento poco evidente. La semantica viene dalla BA, non dalla traiettoria calcolata dal layout engine.

6.4 Esempio Mermaid generico e deterministico

L'esempio mostra uno scope con containment gia' proiettato, un Boundary distinto e una relation qualificata da `crossesBoundary`. La linea continua e' soltanto la projection gia' stabilita per la relation dell'esempio; `crossesBoundary` ne modifica l'enfasi visuale, non il tipo.

```
%%{init: {
  "theme": "base",
  "themeVariables": {
    "background": "#FFFFFF",
    "primaryTextColor": "#17324D",
    "lineColor": "#5B6770",
    "edgeLabelBackground": "#FFFFFF"
  },
  "themeCSS": ".cluster rect { rx:22px !important; ry:22px !important; }"
}}%%

flowchart LR
ExternalStore[("ExternalArtifactStore")]

subgraph ExampleEnvironmentBoundary["ExampleEnvironmentBoundary<br/><span style='
    ↪ font-size:7px'>BAP-EX-B01</span>"]
  direction TB
  subgraph ExampleEnvironment["ExampleEnvironment"]
    direction TB
    LocalStore[("LocalArtifactStore")]
    Service("ExampleService")
  end
end

ExternalStore -->|"crossesBoundary<br/><span style='font-size:7px'>BAP-EX-B02</span
    ↪ >"| LocalStore

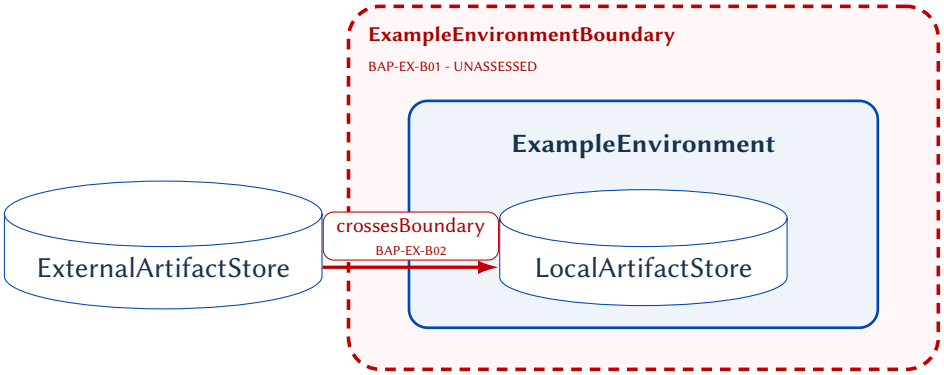
classDef BAReferentD1 fill:#267DFF,stroke:#0047B3,
  stroke-width:2px,color:#FFFFFF;
classDef BAStore fill:#EEF4F9,stroke:#275D86,
  stroke-width:2px,color:#17324D;
class Service BAReferentD1;
class ExternalStore,LocalStore BAStore;

style ExampleEnvironment fill:#F5F9FF,stroke:#0047B3,
  stroke-width:2.5px,color:#17324D
style ExampleEnvironmentBoundary fill:#FFF8F8,stroke:#C62828,
  stroke-width:3px,stroke-dasharray:7 5,color:#8B1A1A
linkStyle 0 stroke:#C62828,stroke-width:3px;
```

Lettura del sorgente

`ExampleEnvironmentBoundary` e' una identity BA distinta da `ExampleEnvironment`. Il frame rosso tratteggiato proietta `boundaryOf`; la relation rossa proietta la relation di base qualificata da `crossesBoundary`. Nessuno dei due colori dichiara trust o rischio gia' valutato. Se la BA identifica un Boundary, non sostituirlo con una cornice generica EXTERNAL o `untrusted`.

6.5 Esempio grafico visivo



Tre letture da tenere separate

Il frame rosso tratteggiato e' la occurrence di `ExampleEnvironmentBoundary`; il frame blu interno continua a essere la projection dello scope `ExampleEnvironment`; la freccia rossa e' una relation gia' esistente cui la BA aggiunge `crossesBoundary`. La sovrapposizione visuale non crea `hasPart` fra Boundary e scope e non riclassifica la relation.

Worst-case visuale, non giudizio di trust

In assenza di security assessment il rosso rende immediatamente visibile il punto che richiede analisi. Una successiva view puo' sostituire l'enfasi cromatica solo quando include una classificazione di sicurezza esplicita e un relativo projection contract; la topology del Boundary e la proposition di crossing restano le stesse.

6.6 Evidence gate, crossing unresolved e regression minima

BOUNDARY-P07 - outside richiede evidenza positiva

Non classificare un elemento come esterno allo scope soltanto perche' non compare nella closure `hasPart` o perche' la documentazione non ne specifica la collocazione. La posizione rispetto al Boundary deve essere supportata da evidenza positiva o da una proposition BA accettata. Assenza di composizione non equivale a "outside".

BOUNDARY-P08 - unresolved non viene trasformato in crossing

Se source, destination o posizione rispetto al Boundary sono `NOT SPECIFIED`, non disegnare una relation rossa di crossing e non collocare artificialmente l'endpoint sul lato esterno. Se il View Contract include i gap documentali, mostrare una card diagnostica separata con `position relative to Boundary = NOT SPECIFIED`; la card non partecipa alla topology del Boundary.

Caso BA	Projection deterministica
`boundaryOf(B,S)`	Frame rosso tratteggiato `B` attorno alla projection di `S`; nessun `hasPart` implicito.
Relation `R` + `crossesBoundary(R,B)`	La projection di `R` conserva il proprio pattern e riceve overlay rosso UNASSESSED; nessuna edge aggiuntiva.
Endpoint noto fuori, endpoint noto dentro, qualifier accettato	I due endpoint restano nei rispettivi lati e la relation qualificata viene evidenziata.
Source/registry `NOT SPECIFIED`	Nessun lato inventato, nessun crossing dedotto; eventuale diagnostic card separata dalla topology.
Security assessment assente	Boundary/crossing restano rossi come worst-case visual treatment UNASSESSED.
Security assessment presente e inclusa nella view	Eventuale override cromatico soltanto secondo il relativo security projection contract; questa guida non inventa giallo/verde o soglie.

Regression gate della projection Boundary

Una implementazione della guida e' conforme soltanto se distingue almeno tre casi: crossing supportato da BA, relation interna senza crossing e crossing non determinabile per evidence incompleta. Il terzo caso non puo' essere reso uguale al primo per prudenza grafica: il worst-case rosso si applica a un Boundary/crossing *identificato ma non security-assessed*, non autorizza a inventare topology mancante.

Checklist operativa

Verificare identity del Boundary e `boundaryOf`; preservare la projection dello scope; applicare il rosso al Boundary non valutato; applicare l'overlay rosso soltanto alle relation qualificate da `crossesBoundary`; non convertire il colore in trust/risk; non usare il routing del renderer come evidence; lasciare `NOT SPECIFIED` quando il lato non e' governato.

7 transfer

VISUAL RULE

7.1 Scopo della rappresentazione

La proposition `transfer` descrive il trasferimento di un contenuto da un `source` a un `destination`. La projection conserva i tre role BA ma non li trasforma in label grafiche. Il contenuto e' una occurrence autonoma e mantiene la propria shape/famiglia visuale.

TRANSFER-P01 - pattern source -> content -> destination

Una `transfer` viene proiettata come percorso direzionale in due segmenti: `source -> content -> destination`. La label `transfer` compare sul primo segmento come nome dell'operatore. Il secondo segmento resta senza label di role. I nomi `source`, `content` e `destination` appartengono alla BA e **non** sono edge label Mermaid canoniche.

```
BAP-EX-T01 - transfer
  source      -> ExampleSender
  destination -> ExampleReceiver
  content     -> ExampleMessage

flowchart LR
  ExampleSender -->|transfer| ExampleMessage
  ExampleMessage --> ExampleReceiver
```

Non serializzare i role come etichette

Non produrre edge label come `source`, `content` o `destination`. La direzione e la posizione dei tre endpoint rendono gia' visibili i role. Una label `destination` sul secondo segmento duplica la struttura BA e introduce rumore grafico senza nuovo significato.

La shape del content non deriva da transfer

`transfer` governa il pattern di connessione, non la forma del contenuto. Un documento, un dataset, un request message o un response envelope mantengono la classe visuale stabilita per la propria natura o per il relativo `semanticKind`. Lo stesso vale per source e destination.

7.2 Request/response come due transfer indipendenti

Un'interazione API puo' essere resa senza introdurre un operatore grafico speciale di "call". Quando la BA governa un request leg e un response leg, la projection mostra due proposition `transfer` distinte e opposte.

TRANSFER-P02 - request e response non vengono fuse

Il request e il response sono due trasferimenti autonomi. Il primo parte dal caller/evaluator e termina sull'endpoint; il secondo parte dall'endpoint e ritorna al caller/evaluator. Non dedurre `invoke`, `initiate`, causalita' interna o protocol behavior ulteriori dal solo fatto che i due transfer appartengano alla stessa interazione applicativa.

```
BAP-EX-T02 - transfer
  source      -> ExampleVerification
  destination -> ExampleEndpoint
  content     -> ExampleRequest

BAP-EX-T03 - transfer
  source      -> ExampleEndpoint
  destination -> ExampleVerification
  content     -> ExampleResponse

flowchart LR
  ExampleVerification -->|transfer| ExampleRequest
  ExampleRequest --> ExampleEndpoint
  ExampleEndpoint -->|transfer| ExampleResponse
  ExampleResponse --> ExampleVerification
```

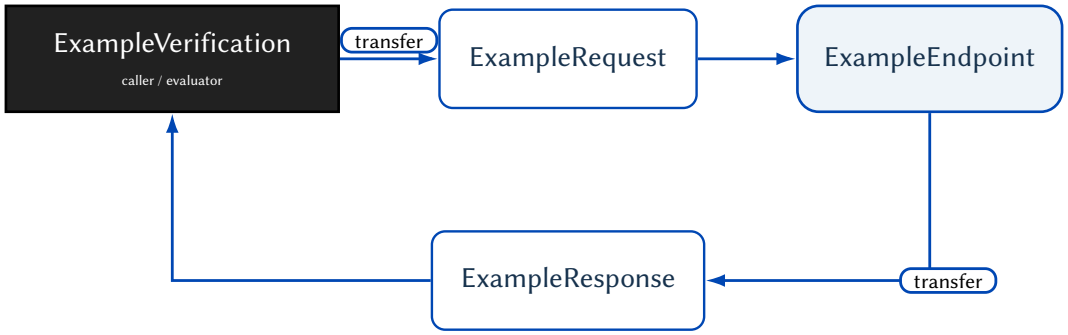
Contenuto parzialmente specificato

La projection puo' mostrare un content BA la cui struttura e' nota solo in parte. Le proprieta' effettivamente governate possono essere annotate nel nodo; quelle non governate restano NOT SPECIFIED. Non si espandono header, body, query parameter, timeout o altri campi soltanto per completare graficamente un request/response.

Composite response

Se la response e' un whole con proposition `hasPart` accettate, si applica la regola di containment gia' stabilita: il response envelope diventa il frame del whole e contiene le part. Il transfer termina sul whole, non su una part scelta per convenienza grafica.

7.3 Esempio grafico generico request/response



Lettura del grafico

Il nodo di verification resta distinto dall'endpoint. Il primo request transfer va dalla verification al request content e poi all'endpoint; il response transfer torna dall'endpoint attraverso il response content. Nessun segmento usa `destination` come label e nessuna relazione interna endpoint-servizi viene dedotta.

8 decisionRule, branch e diagnostica ?

VISUAL RULE

8.1 Decision node e rule card

La semantica di decisionRule resta nella Base Analysis. Questa sezione governa soltanto la sua projection quando il View Contract include esplicitamente la local structure. La decisione viene resa con un rombo; la regola/condizione puo' essere mostrata come card separata per non trasformare il testo della rule nel nome del nodo decisionale.

DECISION-P01 - candidate local structure resta candidate

Se una decisionRule e' CANDIDATE / NOT ADMITTED, la review view puo' visualizzarla soltanto quando la include esplicitamente e deve riportarne lo stato nella rule card. La projection non promuove la local structure a proposition accettata.

```
ReadinessDecision{"condition satisfied?"}
ReadinessRule["decisionRule / branch.when\nCANDIDATE / NOT ADMITTED\n..."]
ReadinessRule -.- ReadinessDecision
```

Association edge della rule card

Il collegamento fra rule card e decision node e' una association di projection, non una nuova proposition BA. Deve restare visivamente secondario e non deve essere letto come flow di esecuzione.

8.2 Stile dei branch: continuo se governato, tratteggiato verso gap

DECISION-P02 - branch governato con linea continua

Quando un branch ha un outcome/result governato nello scope della view, il branch viene reso con freccia continua. La label di branch (per esempio `true` o `false`) puo' essere mantenuta sull'edge.

DECISION-P03 - branch non tracciato completamente verso ?

Quando la decisione implica un branch complementare ma la BA non governa l'outcome o il percorso successivo, il branch termina su un marker `?` projection-only e la freccia e' tratteggiata. Il tratteggio segnala che il percorso deve ancora essere ricostruito; non significa "false", "error" o "FAIL".

```
Decision -->|true| GovernedOutcome
Decision -.->|false| GAP_FALSE
GAP_FALSE(("?"))
```

Non usare il tratteggio per un branch gia' governato

Se anche il branch `false` porta a un outcome o a un ramo BA governato, la sua freccia deve essere continua come quella del branch `true`. Il tratteggio non codifica il valore booleano: codifica soltanto l'incompletezza del percorso nella projection corrente.

8.3 Marker ?: oggetto di projection, non fatto BA

GAP-P01 - il marker ? non appartiene alla BA

Il cerchio rosso `?` e' un marker diagnostico della projection. Non e' un `BAReferent`, non e' una `BAProposition`, non e' un outcome implicito e non puo' essere usato come fonte per reverse inference sulla BA.

GAP-P02 - unresolved semantic relation richiede review evidence

Il pattern `A -> ? -> B` si usa soltanto quando la review ha identificato esplicitamente che fra due fatti gia' esistenti manca una relazione semanticamente rilevante. Il renderer non confronta automaticamente tutte le coppie di nodi e non inventa "relazioni sospette".

```
A -.->|relation NOT SPECIFIED| GAP_REL
GAP_REL -.-> B
GAP_REL(("?"))
```

Linee diagnostiche tratteggiate

I segmenti che collegano un marker `?` restano tratteggiati per distinguerli dal flow governato. Il marker puo' essere accompagnato da una label esplicativa sull'edge, ma la label descrive il tipo di gap e non crea un nuovo operatore BA.

8.4 Missing origin e livello di osservazione

Il missing origin puo' essere rilevato deterministicamente soltanto rispetto al livello di osservazione dichiarato dal View Contract. La presence di un datum in un composite gia' prodotto o trasferito governa la sua provenienza nel contract osservabile, ma non necessariamente la derivazione interna del suo valore.

GAP-P03 - non marcare automaticamente ogni part come origin-missing

Una part contenuta in un whole che arriva attraverso un flow governato non riceve automaticamente un upstream `?`. Se la view mostra soltanto il contract osservabile, il containment e il flow del whole sono sufficienti. Il marker di missing origin compare soltanto quando lo scope richiede anche la derivazione interna del valore e nessuna relation upstream la governa.

```
ExampleResponse
  hasPart -> ExampleStatus

ExampleEndpoint -->|transfer| ExampleResponse
ExampleResponse --> ExampleVerifier

%% Nessun ? -> ExampleStatus nella contract view.
%% Se una internal-origin view richiede chi determina ExampleStatus
%% e la BA non lo governa, allora puo' comparire un diagnostic ?.
```

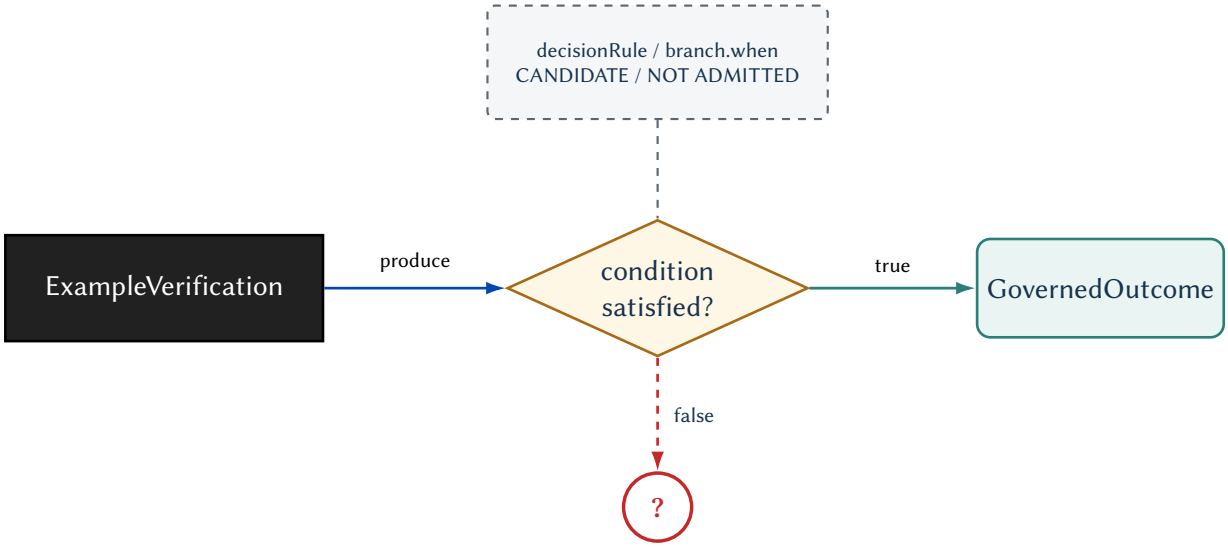
Il diagnostic scompare quando la BA si completa

Quando una successiva analisi introduce una relation upstream governata che spiega l'origine interna del datum, il missing-origin marker viene rimosso automaticamente dalla projection aggiornata. Non serve modificare la BA per cancellare il `?`, perche' il marker non ne ha mai fatto parte.

Non trasformare un gap interno in assenza di response provenance

Il fatto che non sia documentato come un endpoint determini internamente un campo non autorizza a rappresentare quel campo come se "nascesse dal nulla" quando la response envelope e il suo containment sono gia' governati. Contract provenance e internal derivation sono due domande diverse.

8.5 Esempio grafico combinato



Regression minima per decision/gap

Una implementation conforme deve distinguere almeno: (1) branch governato con linea continua; (2) branch complementare non governato con linea tratteggiata verso `?`; (3) rule card candidate separata dalla topology del flow; (4) marker `?` privo di identity BA. Se una di queste differenze viene persa, il grafico non conserva lo stato epistemico della BA/review.